

Program Counter

Test Report

Comprehensive Verification of PolyFlop8 Program Counter

Document ID:	TC-PC-001
Revision:	1.0
Date:	January 5, 2026
Test Level:	Unit/Block Level
Test Engineer:	Artem Horiunov
Status:	PASS

1 TEST OVERVIEW

1.1 Unit Under Test (UUT)

Component	Program Counter for PolyFlop8 Processor
VHDL Entity	ProgramCounter
Source File	ProgramCounter.vhd, tb_ProgramCounter.vhd
Test Environment	ModelSim/QuestaSim 2023.4, VHDL Testbench
Clock Frequency	100MHz (10ns period)
Address Width	11-bit (2048 addresses)

1.2 Scope

This test report documents the verification of the Program Counter for the PolyFlop8 Processor. The Program Counter is responsible for managing the address of the current instruction being executed, supporting sequential execution, jumps, branches, and subroutine calls.

1.3 Test Objectives

- Verify correct operation of the Program Counter during reset
- Validate sequential execution (PC + 1)
- Confirm stall functionality (pc_en = '0')
- Test absolute jump (JMP)
- Test relative branch (positive and negative offsets)
- Test return from subroutine (RET)
- Ensure 100% code coverage of Program Counter behavioral description

2 REQUIREMENTS COVERAGE

Req ID	Requirement Description	Test Case ID	Status
TR-PC-01	Verify Reset Verification	TC-PC-001-01	PASS
TR-PC-02	Verify Fetch Cycle (PC + 1)	TC-PC-001-02	PASS
TR-PC-03	Verify Stall (pc_en = '0')	TC-PC-001-03	PASS
TR-PC-04	Verify Absolute Jump (JMP)	TC-PC-001-04	PASS
TR-PC-05	Verify Branch Positive (+5)	TC-PC-001-05	PASS
TR-PC-06	Verify Branch Negative (-10)	TC-PC-001-06	PASS
TR-PC-07	Verify Return from Subroutine (RET)	TC-PC-001-07	PASS

Table 1: Requirements Coverage Matrix

3 DETAILED TEST CASES

3.1 Test Case TC-PC-001-01: Reset Verification

Requirement: TR-PC-01

Purpose: Verify reset functionality and initial state.

Status: PASS

3.2 Test Case TC-PC-001-02: Fetch Cycle (PC + 1)

Requirement: TR-PC-02

Purpose: Verify sequential execution (PC + 1).

Status: PASS

3.3 Test Case TC-PC-001-03: Stall (pc_en = '0')

Requirement: TR-PC-03

Purpose: Verify stall functionality (pc_en = '0').

Status: PASS

3.4 Test Case TC-PC-001-04: Absolute Jump (JMP)

Requirement: TR-PC-04

Purpose: Verify absolute jump functionality.

Status: PASS

3.5 Test Case TC-PC-001-05: Branch Positive (+5)

Requirement: TR-PC-05

Purpose: Verify relative branch with positive offset.

Status: PASS

3.6 Test Case TC-PC-001-06: Branch Negative (-10)

Requirement: TR-PC-06

Purpose: Verify relative branch with negative offset.

Status: PASS

3.7 Test Case TC-PC-001-07: Return from Subroutine (RET)

Requirement: TR-PC-07

Purpose: Verify return from subroutine functionality.

Status: PASS

4 SOURCE CODES

4.1 ProgramCounter.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ProgramCounter is
    Port (
        clk      : in  STD_LOGIC;

```

```

        rst      : in  STD_LOGIC;
        pc_en    : in  STD_LOGIC;
        pc_src   : in  STD_LOGIC_VECTOR (1 downto 0);
        alu_out   : in  STD_LOGIC_VECTOR (7 downto 0);
        ram_data  : in  STD_LOGIC_VECTOR (7 downto 0);
        jump_abs_11bit : in STD_LOGIC_VECTOR (10 downto 0);
        pc_out   : out STD_LOGIC_VECTOR (10 downto 0)
    );
end ProgramCounter;

architecture Behavioral of ProgramCounter is
    signal pc_reg : unsigned(10 downto 0) := (others => '0');
    signal pc_next_sequential : unsigned(10 downto 0);

begin

    pc_next_sequential <= pc_reg + 1;

    process(clk, rst)
        variable v_offset_extended : signed(10 downto 0);
    begin
        if rst = '1' then
            pc_reg <= (others => '0');

        elsif rising_edge(clk) then
            if pc_en = '1' then
                case pc_src is
                    when "00" =>
                        pc_reg <= pc_next_sequential;

                    when "01" =>
                        v_offset_extended := resize(signed(alu_out), 11);
                        pc_reg <= unsigned(signed(pc_next_sequential) +
                            ↪ v_offset_extended);

                    when "10" =>
                        pc_reg <= resize(unsigned(ram_data), 11);

                    when "11" =>
                        pc_reg <= unsigned(jump_abs_11bit);

                    when others =>
                        pc_reg <= pc_next_sequential;
                end case;
            end if;
        end if;
    end process;

    pc_out <= std_logic_vector(pc_reg);

end Behavioral;

```

4.2 tb_ProgramCounter.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_ProgramCounter is
end tb_ProgramCounter;

architecture Behavioral of tb_ProgramCounter is

```

```

component ProgramCounter
  Port (
    clk          : in  STD_LOGIC;
    rst          : in  STD_LOGIC;
    pc_en        : in  STD_LOGIC;
    pc_src       : in  STD_LOGIC_VECTOR (1 downto 0);
    alu_out      : in  STD_LOGIC_VECTOR (7 downto 0);
    ram_data     : in  STD_LOGIC_VECTOR (7 downto 0);
    jump_abs_11bit : in  STD_LOGIC_VECTOR (10 downto 0);
    pc_out       : out STD_LOGIC_VECTOR (10 downto 0)
  );
end component;

signal clk_tb          : STD_LOGIC := '0';
signal rst_tb          : STD_LOGIC := '0';
signal pc_en_tb        : STD_LOGIC := '0';
signal pc_src_tb       : STD_LOGIC_VECTOR(1 downto 0) := "00";
signal alu_out_tb      : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
signal ram_data_tb     : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
signal jump_abs_11bit_tb : STD_LOGIC_VECTOR(10 downto 0) := (others => '0');
signal pc_out_tb       : STD_LOGIC_VECTOR(10 downto 0);

constant CLK_PERIOD : time := 10 ns;

function to_string(slv : std_logic_vector) return string is
begin
  return integer'image(to_integer(unsigned(slv)));
end function;

begin

  uut: ProgramCounter port map (
    clk          => clk_tb,
    rst          => rst_tb,
    pc_en        => pc_en_tb,
    pc_src       => pc_src_tb,
    alu_out      => alu_out_tb,
    ram_data     => ram_data_tb,
    jump_abs_11bit => jump_abs_11bit_tb,
    pc_out       => pc_out_tb
  );

  clk_process: process
  begin
    clk_tb <= '0';
    wait for CLK_PERIOD/2;
    clk_tb <= '1';
    wait for CLK_PERIOD/2;
  end process;

  stim_proc: process
  begin
    report "TC-PC-01: Reset Verification";
    rst_tb <= '1';
    pc_en_tb <= '0';
    wait for CLK_PERIOD * 2;
    rst_tb <= '0';
    wait for CLK_PERIOD;
    assert pc_out_tb = "000000000000"
      report "Error TC-PC-01: Reset failed. Expected 0, got " & to_string(
        pc_out_tb)
      severity error;
  end process;

```

```

report "TC-PC-02: Fetch_Cycle_(PC_+1)";
pc_en_tb <= '1';
pc_src_tb <= "00";
wait for CLK_PERIOD;
assert pc_out_tb = std_logic_vector(to_unsigned(1, 11))
    report "Error_TC-PC-02: Fetch_step_1_failed.Expected_1,got_" &
        ↪ to_string(pc_out_tb) severity error;

wait for CLK_PERIOD;
assert pc_out_tb = std_logic_vector(to_unsigned(2, 11))
    report "Error_TC-PC-02: Fetch_step_2_failed.Expected_2,got_" &
        ↪ to_string(pc_out_tb) severity error;

report "TC-PC-03: Stall_(pc_en_='0')";
pc_en_tb <= '0';
wait for CLK_PERIOD;
assert pc_out_tb = std_logic_vector(to_unsigned(2, 11))
    report "Error_TC-PC-03: PC_changed_during_stall_state." severity
        ↪ error;
pc_en_tb <= '1';

report "TC-PC-04: Absolute_Jump_(JMP)";
pc_src_tb <= "11";
jump_abs_11bit_tb <= std_logic_vector(to_unsigned(100, 11));
wait for CLK_PERIOD;
assert pc_out_tb = std_logic_vector(to_unsigned(100, 11))
    report "Error_TC-PC-04: JMP_failed.Expected_100,got_" & to_string(
        ↪ pc_out_tb) severity error;

report "TC-PC-05: Branch_Positive_(+5)";
pc_src_tb <= "01";
alu_out_tb <= std_logic_vector(to_signed(5, 8));
wait for CLK_PERIOD;
assert pc_out_tb = std_logic_vector(to_unsigned(106, 11))
    report "Error_TC-PC-05: Branch(+)_failed.Expected_106,got_" &
        ↪ to_string(pc_out_tb) severity error;

report "TC-PC-06: Branch_Negative_(-10)";
pc_src_tb <= "01";
alu_out_tb <= std_logic_vector(to_signed(-10, 8));
wait for CLK_PERIOD;
assert pc_out_tb = std_logic_vector(to_unsigned(97, 11))
    report "Error_TC-PC-06: Branch(-)_failed.Expected_97,got_" &
        ↪ to_string(pc_out_tb) severity error;

report "TC-PC-07: Return_from_Subroutine_(RET)";
pc_src_tb <= "10";
ram_data_tb <= std_logic_vector(to_unsigned(250, 8));
wait for CLK_PERIOD;
assert pc_out_tb = std_logic_vector(to_unsigned(250, 11))
    report "Error_TC-PC-07: RET_failed.Expected_250,got_" & to_string(
        ↪ pc_out_tb) severity error;

report "---_Program_Counter_Tests_Completed_Successfully_---";
pc_en_tb <= '0';
wait;
end process;

end Behavioral;

```

5 SIMULATION LOG

```
Time resolution is 1 ps
Note: TC-PC-01: Reset Verification
Time: 0 ps   Iteration: 0   Process: /tb_ProgramCounter/stim_proc   File: F:/
    ↳ Projects/PolyFlop8-MCU/Polyflop8/Polyflop8.srscs/sources_1/new/
    ↳ Unit_test_bench/ProgramCounter_TB.vhd
Note: TC-PC-02: Fetch Cycle (PC + 1)
Time: 30 ns  Iteration: 0   Process: /tb_ProgramCounter/stim_proc   File: F:/
    ↳ Projects/PolyFlop8-MCU/Polyflop8/Polyflop8.srscs/sources_1/new/
    ↳ Unit_test_bench/ProgramCounter_TB.vhd
Note: TC-PC-03: Stall (pc_en = '0')
Time: 50 ns  Iteration: 0   Process: /tb_ProgramCounter/stim_proc   File: F:/
    ↳ Projects/PolyFlop8-MCU/Polyflop8/Polyflop8.srscs/sources_1/new/
    ↳ Unit_test_bench/ProgramCounter_TB.vhd
Note: TC-PC-04: Absolute Jump (JMP)
Time: 60 ns  Iteration: 0   Process: /tb_ProgramCounter/stim_proc   File: F:/
    ↳ Projects/PolyFlop8-MCU/Polyflop8/Polyflop8.srscs/sources_1/new/
    ↳ Unit_test_bench/ProgramCounter_TB.vhd
Note: TC-PC-05: Branch Positive (+5)
Time: 70 ns  Iteration: 0   Process: /tb_ProgramCounter/stim_proc   File: F:/
    ↳ Projects/PolyFlop8-MCU/Polyflop8/Polyflop8.srscs/sources_1/new/
    ↳ Unit_test_bench/ProgramCounter_TB.vhd
Note: TC-PC-06: Branch Negative (-10)
Time: 80 ns  Iteration: 0   Process: /tb_ProgramCounter/stim_proc   File: F:/
    ↳ Projects/PolyFlop8-MCU/Polyflop8/Polyflop8.srscs/sources_1/new/
    ↳ Unit_test_bench/ProgramCounter_TB.vhd
Note: TC-PC-07: Return from Subroutine (RET)
Time: 90 ns  Iteration: 0   Process: /tb_ProgramCounter/stim_proc   File: F:/
    ↳ Projects/PolyFlop8-MCU/Polyflop8/Polyflop8.srscs/sources_1/new/
    ↳ Unit_test_bench/ProgramCounter_TB.vhd
Note: --- Program Counter Tests Completed Successfully ---
Time: 100 ns Iteration: 0   Process: /tb_ProgramCounter/stim_proc   File: F:/
    ↳ Projects/PolyFlop8-MCU/Polyflop8/Polyflop8.srscs/sources_1/new/
    ↳ Unit_test_bench/ProgramCounter_TB.vhd
```

6 PASS/FAIL CRITERIA

6.1 Success Criteria

Criteria	Description
Functional	All test cases execute without assertion failures
Timing	No timing violations (Program Counter is synchronous)
Coverage	100% statement coverage, 100% branch coverage achieved
Consistency	Identical results across multiple simulation runs
Documentation	All test results properly documented and signed off

6.2 Failure Conditions

- Any single assertion failure during simulation
- Code coverage less than 100% for Program Counter behavioral code
- Timing violations or simulation warnings

- Inconsistent results between simulation runs
- Missing or incomplete test documentation

6.3 Coverage Goals

- **Statement Coverage:** 100% of Program Counter process statements executed
- **Branch Coverage:** 100% of case statement branches taken
- **Condition Coverage:** All branch and jump conditions tested
- **Toggle Coverage:** All signal bits toggled 0→1 and 1→0
- **FSM Coverage:** N/A (Program Counter is not a state machine)

7 SPECIAL TEST CONSIDERATIONS

7.1 Edge Cases

- Test all branches with minimum and maximum offset values
- Verify proper handling of signed and unsigned offsets
- Test boundary conditions for PC and address decoding
- Verify correct behavior during stall and reset
- Test all possible multiplexer combinations

7.2 Timing Considerations

- Program Counter is synchronous - verify setup and hold times
- Monitor for glitches or hazards during input transitions
- Verify stable outputs within specified propagation delay
- Test with minimum, typical, and maximum delay models

7.3 Integration Considerations

- Verify interface compatibility with Control Unit and ALU
- Test handshake signals with Control Unit
- Validate output format matches processor expectations
- Confirm timing compatibility with other processor components